Calibrated Reliability-Aware RGB Map Relocalization for GPS/GNSS-Denied UAV Navigation

Version: v6 (28 September 2026)
Optional local extension: Turkish/HGM geospatial transfer study
Project type: Undergraduate graduation/capstone research project
Planned duration: 7 months
Primary domain: Computer vision, visual geo-localization, uncertainty calibration, edge computing, UAV navigation simulation
Core hardware: NVIDIA Jetson Orin Nano + workstation
Core software: Python, PyTorch, OpenCV, Rasterio/GDAL, NumPy/Pandas, scikit-learn; optional ONNX/TensorRT; optional PX4 SIH
Mandatory data source: at least one public UAV visual-localization benchmark with real UAV query images, georeferenced reference maps and sequential ground truth
Real UAV flight: not required
HGM data: not required; optional Turkish-domain transfer/proxy study

Minimum completion criterion (independent of flight/hardware extensions): A reproducible, dataset-driven GPS/GNSS-denied navigation simulation in which:

- real UAV query images from a public benchmark are matched to a georeferenced map;
- the resulting absolute fixes are assigned raw, heuristic, chi-square-consistency and calibrated reliability scores;
- these fixes are used to correct a deliberately drifting position estimate under identical drift seeds, across a sweep of relocalization call rates, including a large-drift reacquisition test;
- the selected RGB localization/reliability pipeline runs on Jetson Orin Nano.

No real flight, local UAV imagery, HGM purchase, ROS 2, PX4, or 3D simulator is required. Completion means these experiments are carried out correctly and reported, whatever their outcome; it does **not** require the proposed method to outperform the baselines.

---

1. Project Summary

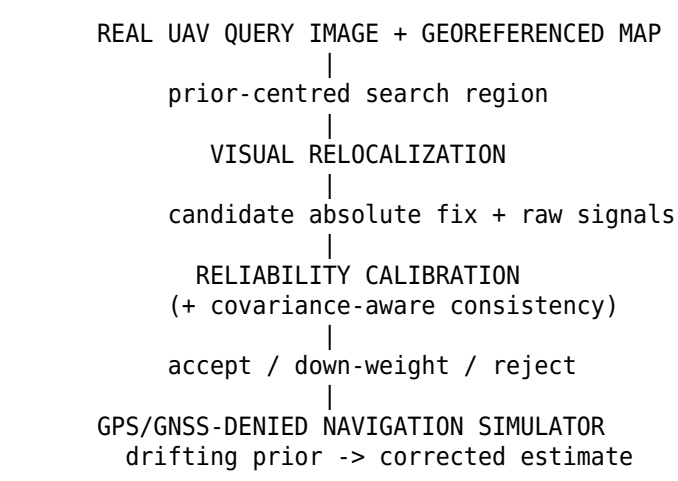
GPS/GNSS-denied UAV navigation needs:

1. a **local/relative estimate** that evolves continuously but drifts; and
2. an **absolute map-based relocalization** that occasionally corrects the drift.

This project focuses on the **reliability of the absolute visual fix**. A public UAV dataset provides real RGB UAV images, georeferenced satellite/aerial maps, and position ground truth. The team implements a practical UAV-to-map localization pipeline, then inserts its fixes into a controlled drift simulator.

The central question:

> **Can a calibrated, validated reliability score distinguish trustworthy from harmful map-based absolute fixes better than raw matching scores, a published heuristic confidence, and standard chi-square innovation gating — and does this improve sequential GPS/GNSS-denied navigation, including recovery from large drift?**



If Turkish/HGM data are obtained, they plug into the same map interface as a **transfer/proxy study**. Without real UAV query imagery over the HGM area, HGM experiments must be described as map-domain transfer or synthetic-view experiments, never as real-flight validation.

Operational map assumption. All georeferenced reference imagery required for the planned area of operation is assumed to be loaded onto the Jetson before the mission, together with any precomputed tile descriptors/indexes. Runtime localization does not require Internet, cellular, satellite, or ground-station communication; map retrieval refers to local retrieval from onboard storage.

---

2. Why the Project Is Framed This Way

2.1 What is already established

- coarse-to-fine UAV-to-map localization;
- retrieval followed by local matching and geometric verification (homography/PnP/affine);
- sequence-aware localization;
- map-aided visual-inertial fusion (e.g., GeoVINS);
- confidence-weighted fusion with rejection of weak absolute fixes (e.g., SatLoc-Fusion, which maps hand-designed confidence into Kalman-filter noise and drops low-confidence absolute fixes);
- quality gating and local map relocalization after large drift: a 2026 orthophoto-aided VO system already gates map observations using matches, inliers, reprojection error and correction displacement, then searches neighbouring map regions when the prior is poor;
- continuous orthophoto/DSM anchoring (e.g., OrthoTrack, 2026);
- chi-square innovation gating — the textbook Kalman-filter method for rejecting inconsistent measurements.

Implementing `UAV image → map match → EKF correction`, heuristic quality gating, or a large-drift relocalization search is **not** a contribution by itself.

2.2 Specific research gap

> **Does the confidence produced by a localization pipeline actually predict whether the resulting fix is correct enough to use — is it calibrated — and does calibrated image-based reliability add value beyond covariance-based chi-square gating in sequential navigation?**

Hand-designed confidence (e.g., SatLoc-style sigmoid combinations) is not validated as a calibrated probability. Recent orthophoto-aided systems also use useful but heuristic quality gates and local relocalization. Chi-square gating uses only the prior covariance, not image evidence, and can fail when the covariance is wrong. Therefore the defensible contribution here is **not gating or reacquisition alone**; it is calibrated fix reliability, explicit failure-prediction evaluation, and evidence of whether image-derived reliability adds value beyond heuristic and covariance-only gates in sequential navigation. This project measures:

- whether confidence predicts fix correctness;
- whether it is calibrated;
- whether calibrated reliability improves navigation outcomes relative to both heuristic confidence and chi-square gating;
- behaviour after large drift, map appearance changes and viewpoint/scale differences;
- the compute cost of the reliability mechanism on Jetson Orin Nano.

2.3 HGM is valuable but cannot be the mandatory query source

HGM orthophoto/DSM/satellite data are high-quality Turkish reference maps. Without an independent UAV flight over the same area, however, they do not provide real camera-view queries. Therefore:

- **public UAV datasets carry the mandatory validation**;
- **HGM is an optional transfer/proxy study**;
- synthetic HGM-derived views are clearly labelled integration tests.

---

3. Research Questions

RQ1 — Confidence validity. How well do common visual-localization confidence signals predict whether an RGB map-derived fix is within the predefined acceptable error?

RQ2 — Calibration vs heuristic. Does a lightweight post-hoc calibrated reliability model improve failure prediction, risk–coverage, Brier score, and calibration error over raw scores and a reproduced published heuristic?

RQ3 — Navigation benefit. Under identical drift seeds and at each relocalization call rate, does calibrated reliability reduce harmful corrections and trajectory error compared with unconditional correction, heuristic gating, and chi-square innovation gating? Does combining calibrated reliability with chi-square gating help?

RQ4 — Large-drift recovery. Can the system recover after the prior becomes substantially wrong — accepting a correct distant fix without accepting a false one?

RQ5 — Edge deployment. What latency, memory, power and compute trade-offs arise when the pipeline runs on Jetson Orin Nano?

Optional RQ6 — Turkish/HGM transfer. If HGM data are obtained, how does reliability calibration shift with map provider, date, resolution or modality?

---

4. Working Hypotheses

- **H1:** Raw retrieval/matching scores are imperfect predictors of meter-level correctness.

- **H2:** Lightweight calibration of geometric and retrieval signals improves reliability estimation without a heavy network.
- **H3:** Reliability-gated correction reduces catastrophic jumps compared with accepting every fix.
- **H4:** Chi-square gating fails in characteristic ways — it rejects correct fixes when the covariance under-represents true drift, and accepts confident-but-wrong fixes near the prior. Image-based reliability can complement it.
- **H5:** Raw prior displacement is a misleading consistency feature after large drift. Covariance-aware consistency plus explicit reacquisition with confirmation is required.
- **H6:** Calibration learned on one dataset/map domain degrades after domain change (AnyVisLoc, HGM).

A negative result is acceptable. Success requires a correct, reproducible evaluation, not confirmed hypotheses.

---

5. Scope and Explicit Exclusions

In scope

- RGB UAV-to-map visual localization;
- retrieval + local matching/geometric verification;
- fix-level confidence and calibration;
- chi-square innovation gating baseline;
- controlled drift simulation;
- sequential fix gating;
- large-drift reacquisition with confirmation;
- Jetson profiling;
- public UAV datasets;
- optional HGM transfer;
- optional synthetic rendering from orthophoto + DSM;
- optional PX4 SIH demo.

Out of scope for mandatory completion

- real UAV flight;
- new local UAV dataset;
- new VIO/SLAM;
- GNSS jamming/spoofing hardware;
- RL navigation; swarms;
- new large network architectures;
- path planning;
- mandatory ROS 2/Gazebo;
- a relocalization **scheduler** (see M10 — optional extension only);
- runtime downloading/requesting of satellite or orthophoto imagery during flight;
- presenting synthetic HGM views as real validation.

Operational Map Assumption and Map Availability

The operational concept assumes that the georeferenced reference imagery for the planned area of operation (AO) is preloaded onto the Jetson before flight, together with any precomputed retrieval descriptors/indexes required by the selected localization pipeline. Runtime localization therefore does not depend on an Internet, cellular, satellite, or ground-station data connection.

This introduces finite-map limitations that must be treated explicitly. Localization performance depends on the coverage, age, spatial resolution, provider/domain, and onboard storage footprint of the preloaded map. If the UAV leaves the mapped AO, if accumulated drift causes the true position to fall outside the searchable portion of the stored map, or if the imagery is too stale or visually inconsistent, absolute map-based relocalization may become unavailable or unreliable. Such conditions must be detected and reported separately from ordinary matching failures.

For deployment, a practical strategy is to preload the planned route plus a safety/reacquisition margin. A coarse larger-area map and higher-resolution tiles for the expected mission corridor may also be used, but hierarchical map storage is an optional extension rather than a mandatory contribution.

---

6. Mandatory Requirements — “Must Do” Items

M1. Public real-UAV benchmarks — roles fixed

Dataset	Role	Why	Notes
AerialVL (primary)	RQ1–RQ5; **the sequential simulator runs on it**	11 real UAV sequences (3.7–11 km, ~70 km total) with satellite imagery and GNSS ground truth; DEM released for the area	Verify IMU/altitude fields and license in Week 1
AnyVisLoc (secondary)	RQ1–RQ2 calibration and **cross-dataset transfer**; not sequential	Real UAV images, multi-altitude/ multi-view, with aerial-orthophoto	Not continuous trajectories. Non-commercial academic licence: no

Dataset	Role	Why	Notes
		and satellite 2.5D reference maps, each with a DSM. The paper reports ~18,000 images; the current public release contains 20,077 images across 24 scenes	redistribution, mirroring or public release of derived datasets without permission. Its satellite DSM is ~30 m; only the aerial maps are an HGM-style (orthophoto + DSM) surrogate
UAV-VisLoc (fallback)	Sequential fallback if AerialVL fails; compact cross-dataset check	6,742 drone images along flight tracks, 11 satellite maps	—
OrthoLoC (optional)	Optional surrogate for national-agency-style orthographic geodata	UAV images with orthophoto + DSM geodata from Germany/US	Verify availability, license and venue in Month 1
SatLoc	Paper baseline only	The dataset release was not confirmed at the time of writing	Use the data only if a release is verified

Fallback order for the sequential core: AerialVL → UAV-VisLoc. AnyVisLoc cannot replace the sequential core.

Verify licenses before redistributing any subset or derivative.

M2. Stable visual localization baseline

One clear pipeline:

```
query image
  -> coarse retrieval over tiles inside the prior-centred search region
  -> local matching on top-K candidates
  -> geometric verification (RANSAC homography/affine or PnP with DSM)
  -> metric map coordinate fix
  -> raw confidence signals
```

Recommended components:

- a published global descriptor or dataset baseline;
- SuperPoint + LightGlue or XFeat + LightGlue;
- RANSAC.

A second pipeline is optional.

This pipeline is **assembled by the students** from published components. It is not a reproduction of the AerialVL paper's algorithm; the AerialVL repository is primarily a data release. The AnyVisLoc repository provides runnable retrieval → matching → PnP baseline code (CAMP retrieval, RoMa matching), which may be used as an implementation reference, subject to its licence. RoMa is a dense matcher and may be slow on the Jetson; the recommended LightGlue-based matchers are lighter.

M3. Ground-truth-only good/unsafe label — fixed now

```
fix_error_i = || predicted_position_i − ground_truth_position_i ||   (metres, 2D)

PRIMARY    unsafe fix:  fix_error > 10 m
SECONDARY  unsafe fix:  fix_error > 25 m     (shared with the thermal project)
SENSITIVITY:            5 m, 50 m            (report only)
```

Justification of 10 m:

- **Containment:** an accepted fix must leave the true position well inside the next prior-centred search region. 10 m is small relative to the minimum search radius (M7).
- **Resolution:** at 100–300 m flight altitude, current pipelines often reach metre-level accuracy (the AnyVisLoc benchmark baseline places 74.1% of its images within 5 m). A 25 m threshold would label nearly every fix "safe" and leave the reliability study with no resolution.

The threshold is declared in this proposal and must not be changed after results are seen. Only gate thresholds may be tuned, on validation data.

M4. Reliability features without label leakage

Candidate runtime features:

- top-1 retrieval score;
- top-1 – top-2 margin;
- retrieval score entropy/concentration;
- number of matches;
- RANSAC inlier count and inlier ratio;
- reprojection error;

- inlier spatial coverage/dispersion;
- transform condition/degeneracy indicators;
- query/reference scale ratio;
- altitude/attitude metadata if available at runtime (see M5);
- **Mahalanobis distance of the fix from the prior** (covariance-aware consistency).

Never use ground-truth error or any quantity derived from the label. Plausibility and geometry indicators are **features**, never part of the label.

Ablate:

- visual-only calibrated reliability;
- visual + Mahalanobis calibrated reliability.

This separates the image-evidence contribution from the covariance contribution.

M5. Confidence baselines

1. **Raw confidence** — one direct score (e.g., inlier ratio or top-1 similarity).
2. **Heuristic confidence (SatLoc-style), reimplemented from the paper's equations:**
   - C1 = w_s·C_score + w_r·C_res;
   - C_score is a sigmoid of the top-1 vs mean-suboptimal retrieval-score margin minus a threshold;
   - C_res is a sigmoid of the UAV/satellite GSD ratio minus 0.8.

   Notes for faithful reproduction:

   - The published equations are internally inconsistent: the sign of the score term, and the term names used in the combining equation. Implement the evidently intended monotonic form (confidence increases with score margin and with GSD ratio) and document the choice.
   - Parameters not stated in the paper (score threshold, weights) are tuned on validation data only.
   - The GSD term needs altitude. Use ground-truth altitude above ground (AerialVL GNSS height minus the released DEM) plus Gaussian noise ($\sigma \approx 2$ m) as a **barometer surrogate**, and document it. If altitude cannot be derived for a dataset, drop the GSD term there and document it rather than improvising.
3. **Chi-square innovation gate** (M8) — the covariance-only baseline.
4. **Calibrated reliability** — logistic regression, isotonic/Platt calibration, or a shallow gradient-boosted model. A tiny MLP only if simpler models fail.

M6. Controlled GPS/GNSS-denied drift simulator

Use the AerialVL ground-truth trajectory and inject controlled drift:

- constant bias;
- velocity/scale bias;
- Gaussian random walk;
- slowly varying heading bias;
- occasional burst error.

Requirements:

- **mild / moderate / severe** regimes;
- the filter's assumed process noise must be configurable **separately** from the true drift, to test covariance mis-specification (filter assumes "mild", truth is "severe");
- every run records its seed and parameters.

M7. Prior-centred search and offset-dependent fixes

Visual-localization difficulty depends on how wrong the prior is, because the search region is centred on it.

Search radius:

Define the largest 1-sigma positional uncertainty as

```
σ_pos = sqrt(λ_max(P_prior))
```

where `λ_max` is the largest eigenvalue of the 2-D position covariance. This keeps the units in metres.

```
NORMAL mode:     r = clip(3·σ_pos, r_min, r_max)   e.g. r_min = 100 m, r_max = 1000 m
REACQUIRE mode:  r = r_max (or whole map)
```

Record both the **true offset** (|prior – ground truth|, evaluation only) and the **chosen search radius**. If the true position lies outside the nominal search region (`in_window = false`), flag the case explicitly and report it separately. Do **not** hard-code `in_window = false` as the unsafe-fix label: the safety label remains the actual ground-truth fix error from M3, and map tiles that overlap a search boundary can create edge cases.

Implementation options (both deterministic):

- **Preferred — exact tile-level cache.** Retrieval depends on the prior only through which tiles are eligible. So:
  1. compute each query's global-descriptor similarity to **every** map tile once;
  2. at run time, restrict to tiles within r of the prior and take the top-K;
  3. run local matching + verification for each (query, tile) pair the first time it is needed, and cache the result.

  The simulator then reproduces exactly what the online localizer would output for any prior and radius, at a fraction of the cost.

- **Fallback — fixed offset grid.** If caching is impractical:
  - precompute each query on a fixed grid of 8 directions (every 45°) × magnitudes {25, 50, 100, 200, 400, 800} m, at each search radius the policies can choose;
  - at run time, select the record at the **nearest grid magnitude and direction** to the actual prior-error vector, and at the radius actually chosen;
  - no random sampling at run time.

In both options the true offset is used only inside the simulator to produce the observation. It remains **[eval]** and is never passed to any gate.

Build the cache/grid for **train/validation sequences only** during development. Build it for **test sequences once, after the configuration freeze** (end of Month 5; see §9.1).

M8. Reliability-aware correction policy and chi-square baseline

Proposed policy:

```
RELIABLE    -> accept with nominal measurement covariance R
UNCERTAIN   -> down-weight (R scaled by a fixed, documented function of reliability)
UNRELIABLE  -> reject
```

Mandatory chi-square innovation baseline:

$$\nu = z_{fix} - H\,\hat{x}_{prior}; \quad S = H\,P_{prior}\,H^{T} + R_{nominal}; \quad d^2 = \nu^{T} S^{-1} \nu$$

reject if $d^2 > \chi^2_m(1-\alpha)$ $\quad m = \dim(z) = 2$; $\alpha = 0.01 \to 9.21$; $\alpha = 0.05 \to 5.99$

This is the general Kalman-filter form. In the mandatory simulator the state is 2-D position only, so H = I and S = P_prior + R_nominal. If velocity or heading is added to the state, H selects the position components and the general form must be used.

$R_{nominal}$ is estimated from **validation-split** localization errors of fixes that satisfy the primary ground-truth label (M3), not from fixes accepted by any policy under evaluation.

Evaluate:

- chi-square alone;
- calibrated reliability alone;
- **combined** (both must pass; down-weight per reliability).

M9. Reacquisition logic with confirmation

A correct fix may lie far from a badly drifted prior. Raw displacement must never automatically make it "bad", and relaxed consistency must never make a false fix easy to accept.

Mandatory rule:

1. In NORMAL mode, use covariance-aware (Mahalanobis) consistency, never raw Euclidean displacement.
2. After N consecutive rejected/missing fixes, or when σ_{pos} exceeds a set bound, enter **REACQUIRE**:
   - widen the search to r_max;
   - suspend the chi-square/consistency penalty.
3. In REACQUIRE, accept a fix only after **confirmation**: k (default 2) fixes, each passing the reliability gate, whose **relative displacement agrees with the dead-reckoned displacement between them** within a tolerance τ.
   - This works because drift is low-frequency, so short-interval relative motion from the drifting estimate stays accurate even when absolute error is large.
   - Suggested τ: 2 × primary threshold + expected relative drift over the interval.
4. Return to NORMAL after the confirmed fix is applied.

Mandatory test scenarios (severe drift):

- **Scenario A — large drift + correct distant fix:** evaluate whether, and under what conditions, REACQUIRE restores localization, and how long it takes.
- **Scenario B — large drift + plausible false fix:** inject a confident wrong fix (e.g., from a self-similar region) and measure the false-accept rate, compared with the baselines.

Report the **false-accept rate during REACQUIRE separately**, together with recovery time. As everywhere in this project, the requirement is to run and report these tests correctly, not to obtain a particular outcome.

M10. Fair baselines, evaluation references, and the budget axis

Compare at minimum:

1. **Drift only.**
2. **Unconditional correction.**
3. **Raw-score gate.**
4. **Heuristic (SatLoc-style) gate.**
5. **Chi-square innovation gate.**
6. **Calibrated gate.**
7. **Calibrated + chi-square (combined).**
8. **Randomized-decision reference** — an uninformative control. For each call-rate × drift-regime × filter-setting stratum, estimate the calibrated policy's distribution of `(decision, R-scale)` tuples on **validation data only**. Freeze that distribution before the test run. On test sequences, draw a tuple independently at each attempted fix using a fixed seed. This approximately matches the intervention intensity of the calibrated policy without using image evidence or test-set outcomes.
9. **Label-oracle reference** — uses the primary ground-truth safety label to accept safe fixes and reject unsafe fixes. This is a **classification reference/ceiling**, not a formal upper bound on trajectory error: a "safe" fix can still worsen an already-accurate prior, and an "unsafe" fix can sometimes reduce a very large prior error.

The randomized-decision and label-oracle references are **evaluation-only** controls; neither is a deployable policy. At a given call rate, all policies and references use the same attempted localization times and drift seeds.

Budget axis — important clarification: this project has **no scheduler**. Every gate policy runs the localizer on the same call schedule, so the number of calls is matched automatically, and rejecting a fix saves no compute. The budget axis is therefore a **call-rate sweep**: every policy is evaluated at fixed relocalization intervals (e.g., every 1, 2, 5, 10, 20 s of flight, or the equivalent frame spacing). Report error vs call rate for all policies.

An uncertainty-triggered scheduler (as in the thermal project) is an **optional extension** (O6), not part of the core.

M11. Jetson Orin Nano deployment

Run query localization + reliability on the Jetson. Measure:

- end-to-end latency;
- retrieval time;
- matching/verification time;
- reliability overhead;
- memory;
- GPU/CPU utilization;
- temperature;
- power;
- energy per call where measurable.

No minimum FPS required. TensorRT is desirable but not mandatory; PyTorch/ONNX Runtime on the Jetson is an acceptable fallback. Re-check calibration after any backend/precision change.

M12. Reproducible final package

- repository;
- pinned environment;
- dataset preparation;
- map tiling/georeferencing;
- drift simulator;
- fix-record generator;
- calibration scripts;
- chi-square and SatLoc-style baselines;
- call-rate experiment runner;
- Jetson profiling;
- configs/seeds;
- CSV/JSON logs;
- report;
- paper-style manuscript;
- demo video (query → map fix → simulated drift correction, including a reacquisition event).

---

7. Common Fix-Record Interface (shared with the thermal project)

One record per attempted fix (JSONL or CSV). Fields marked **[eval]** must **never** be read by any deployable policy.

```
# run identity / reproducibility
```

```
run_id, policy_id, seed, drift_regime, filter_noise_setting, call_interval_s
sequence_id, query_id, timestamp_s
magnitude_idx, direction_idx # grid fallback only: nearest offset-grid point used
mode                         # NORMAL / REACQUIRE / LOST (project-dependent; RGB normally uses NORMAL/RE

# state
ground_truth_x_m, ground_truth_y_m                  [eval]
prior_x_m, prior_y_m
prior_cov_xx, prior_cov_xy, prior_cov_yy

# localization geometry
true_offset_m                # |prior − ground truth|                  [eval]
offset_bin                   # if precomputed-bin fallback is used
search_window_m              # search radius actually used
in_window                    # true position inside searched region    [eval]

# fix
predicted_fix_x_m, predicted_fix_y_m
raw_confidence
heuristic_confidence         # SatLoc-style
calibrated_reliability
chi2_d2                      # νᵀS⁻¹ν
confirmation_count           # REACQUIRE only
fix_error_m                                         [eval]
safe_primary, safe_secondary                        [eval]
decision                     # accept / down-weight / reject / pending-confirmation

# cost
localization_latency_ms, confidence_latency_ms
power_w, energy_per_fix_j    # if measured
```

---

8. Recommended Datasets and Resources

8.1 AerialVL — primary

Use for:

- the main localization baseline;
- real fix errors;
- the sequential drift-correction simulator;
- calibration.

Its released DEM enables altitude above ground for the heuristic baseline's GSD term.

8.2 AnyVisLoc — secondary (calibration/transfer)

Use for RQ1–RQ2 calibration and transfer (calibrate on AerialVL, test on AnyVisLoc, and vice versa). Low-altitude, oblique and multi-view images stress the confidence signals.

Constraints:

- the current public release has 20,077 images across 24 scenes (the paper's numbers refer to its ~18,000-image version);
- non-commercial academic licence: no redistribution or public release of derived datasets without permission;
- not sequential;
- its satellite DSM (~30 m) is too coarse for precise pose work; use the aerial DSM for any DSM-related analysis.

8.3 UAV-VisLoc — fallback/compact

Sequential fallback and compact cross-dataset check.

8.4 OrthoLoC — optional

Optional HGM-like surrogate (orthographic geodata + DSM). Verify availability before planning around it.

8.5 SatLoc — paper baseline

Use for the heuristic-confidence formulation (M5). Do not depend on its data.

8.6 Turkish/HGM data — optional transfer study

If obtained:

- orthophoto → reference map;
- SYM5-L0 DSM → optional terrain geometry;
- Göktürk-2 → lower-resolution cross-provider map.

No-flight experiments:

1. **Map-date/provider shift:** calibrate on public data, then evaluate confidence shift when matching between Turkish map sources/dates (map-to-map proxy, labelled as such).
2. **Synthetic camera proxy:** Blender renders from orthophoto + DSM, labelled synthetic.
3. **Coarse-to-fine provider experiment:** Göktürk-2 → orthophoto.

Product terms, pricing and university-access rules change. Re-check the HGM product pages at request time and record the date. Never place HGM source imagery in a public repository unless redistribution is explicitly permitted.

---

9. Recommended Methodology

9.1 Phase A — Freeze benchmark and map interface

1. Download AerialVL.
2. Verify the license and available fields (GNSS, altitude, IMU, DEM).
3. Define one metric CRS convention.
4. Build the tile index.
5. Plot ≥20 queries over the map.
6. Freeze train/validation/test splits by **sequence** (no sequence in more than one split). All calibration, gate thresholds, SatLoc-style parameters, `R_nominal` and REACQUIRE parameters are set on train/validation sequences only. **During Months 1–5, test sequences are not processed at all:** no cache, no fix records, no simulator runs.

Configuration freeze (end of Month 5). A tagged repository commit fixes all code, configuration files and tuned parameters:

- the localization pipeline;
- calibration models;
- gate thresholds;
- SatLoc-style parameters;
- `R_nominal`;
- REACQUIRE parameters (N, k, τ).

Nothing in it may change afterwards in response to test results.

Held-out test run (Month 6). Only after the freeze, generate the **test-sequence** tile cache / fix records **once** and run the held-out evaluation.

If a genuine bug is found after the test run, fix it, document the fix and the affected results in the report, and rerun. Never re-tune on test results.

9.2 Phase B — Visual localization baseline

For each query:

1. Retrieve within the prior-centred region.
2. Take the top-K candidates.
3. Run local matching.
4. Run geometric verification.
5. Convert to metric coordinates.
6. Log all raw signals.

9.3 Phase C — Fix-reliability dataset

Per query and offset condition, record:

- the fix;
- ground truth;
- error;
- retrieval, match and geometry statistics;
- metadata;
- `in_window`.

Labels come from ground-truth error only (M3).

9.4 Phase D — Confidence baselines and calibration

```
raw score -> SatLoc-style heuristic -> calibrated (visual-only) -> calibrated (visual + Mahalanobis)
```

Train/tune on training/validation splits only. Evaluate:

- AUROC/AUPRC;
- Brier score;
- reliability diagrams/ECE;
- risk–coverage;
- false accepts of unsafe fixes.

Also evaluate cross-dataset (AnyVisLoc) with frozen models: first without re-fitting, then with re-calibration only. For re-calibration, fit on a scene-disjoint AnyVisLoc subset and report on the remaining scenes.

9.5 Phase E — Deterministic navigation simulator

Inputs:

- ground-truth trajectory;
- drift model;
- filter noise setting;
- fix database or online localizer;
- gate policy;
- call interval.

Outputs:

- drifting prior;
- decisions;
- corrected trajectory;
- error over time;
- mode timeline;
- compute totals.

Plain Python; runs headless in minutes.

9.6 Phase F — Large-drift recovery

Construct the two scenarios of M9 under severe drift and mis-specified filter noise. Measure:

- whether the chi-square gate rejects the correct distant fix (expected, but to be tested);
- whether, and how quickly, REACQUIRE + confirmation recovers;
- how often each policy accepts the injected false fix.

9.7 Phase G — Call-rate sweep

For every drift seed and every policy (M10), run each call interval. Report:

```
trajectory RMSE / max error      vs  call interval
harmful-correction count         vs  call interval
localization compute per sequence (Jetson-measured cost × calls)
```

All policies at the same interval use the same calls, so any difference comes from the gate.

9.8 Phase H — Jetson deployment

After the pipeline is frozen:

1. PyTorch/ONNX on the Jetson;
2. profile;
3. optimize the bottleneck stage only;
4. TensorRT where supported;
5. re-check calibration.

9.9 Optional Phase I — HGM synthetic/transfer

Blender scripted rendering:

```
HGM orthophoto (texture) + HGM DSM (terrain mesh)
    + camera trajectory (Blender Python)
    -> rendered RGB views with exact synthetic poses
```

Results are labelled **synthetic HGM proxy**.

9.10 Optional Phase J — PX4 SIH demonstration

Headless PX4 internal simulator for a flight-stack demo, fed by the simulator's correction decisions. Must not delay core work.

---

10. Evaluation Metrics

Visual localization:

- median/mean 2D error;
- recall within 5/10/25/50 m;
- catastrophic-error rate;
- retrieval Recall@K;
- `in_window` rate vs true offset.

Reliability/calibration:

- AUROC/AUPRC (primary and secondary labels);
- Brier score;
- ECE/reliability diagrams;
- risk–coverage;
- false-accept and false-reject rates;
- gap to the label-oracle reference;
- cross-dataset calibration shift.

Sequential navigation:

- ATE/2D RMSE;
- final and maximum error;
- harmful-correction count;
- recovery time after large drift;
- REACQUIRE false-accept rate;
- fraction of time in REACQUIRE;
- chi-square false rejects of correct fixes.

Embedded:

- latency per stage;
- reliability overhead;
- compute per sequence;
- GPU duty time;
- memory;
- temperature;
- power;
- energy per call.

Do not translate Jetson compute savings into flight-endurance claims.

---

11. Minimum Experiment Matrix

All simulator rows are run at ≥4 call intervals × 3 drift regimes × {correct, mis-specified} filter noise, with identical seeds across policies.

ID	Gate	Mode logic	Data	Purpose
E1	none (drift only)	—	AerialVL	lower reference
E2	accept all	—	AerialVL	unconditional baseline
E3	raw score	NORMAL	AerialVL	raw baseline
E4	SatLoc-style heuristic	NORMAL	AerialVL	published-heuristic baseline
E5	chi-square	NORMAL	AerialVL	covariance-only baseline
E6	calibrated (visual-only)	NORMAL	AerialVL	image evidence alone
E7	calibrated (visual + Mahalanobis)	NORMAL	AerialVL	proposed
E8	calibrated + chi-square	NORMAL	AerialVL	combined
E9	randomized decision reference	NORMAL	AerialVL	uninformative decision control
E10	label oracle	NORMAL	AerialVL	ground-truth classification reference
E11	proposed / combined	NORMAL + REACQUIRE + confirmation	AerialVL, severe drift, Scenarios A and B	large-drift recovery
E12	calibrated reliability (frame-level)	—	AnyVisLoc	calibration transfer

ID	Gate	Mode logic	Data	Purpose
E13	selected	selected	AerialVL subset	Jetson profiling

Optional:

- **E14:** HGM transfer or synthetic HGM rendering;
- **E15:** OrthoLoC transfer;
- **E16:** triggered scheduler.

---

12. Equipment and Fallback Plan

Resource	Intended use	Mandatory?	Fallback
Workstation/GPU	Preprocessing, localization, batch experiments	Yes	Pretrained models, reduced evaluation subset
Jetson Orin Nano	Edge deployment and profiling	**Yes**	PyTorch/ONNX if TensorRT fails
Public UAV benchmark	Real queries + map + ground truth	**Yes**	AerialVL → UAV-VisLoc (sequential); AnyVisLoc for calibration/transfer
HGM orthophoto/DSM	Turkish transfer/proxy	No	Project complete on public data
UAV	Real flight	No	Never required
IMU/Pixhawk	Real VIO/HIL	No	Controlled drift simulator
ROS 2 / Gazebo	Demo only	No	Python simulator; Blender for rendering; PX4 SIH for flight stack
Preloaded reference map + tile/index database	Offline map-based localization during operation	Yes conceptually	Reduce AO, map resolution, or number of stored feature products; benchmark experiments use locally stored dataset maps

Fallback hierarchy

```
Level 1: AerialVL + workstation visual localization
    -> Level 2: reliability calibration + chi-square/heuristic baselines + drift simulator
    -> Level 3: reacquisition test + Jetson profiling
    -> PROJECT SUCCESS
    -> Level 4: AnyVisLoc/OrthoLoC transfer, optional HGM, Blender views
    -> Level 5: optional PX4 SIH / scheduler
    -> Level 6: optional future real flight
```

---

13. Simulation Strategy

Mandatory: a custom deterministic Python dataset-replay simulator.

Optional:

- PX4 SIH — headless flight-stack demo;
- Blender — rendering views over orthophoto + DSM (a renderer, not a flight simulator);
- jMAVSim — simple visualization only.

Not recommended as core dependencies: Gazebo, the original Microsoft AirSim (archived), Flightmare, Isaac Sim.

---

14. Seven-Month Work Plan

Month 1 — Feasibility and reproducibility

Tasks:

- read the priority papers;
- download AerialVL and verify fields/license;
- plot queries on the map;
- implement the fix-record schema;
- copy the M3 labels into the README;
- write the first drift generator;
- implement the chi-square gate with a unit test;
- verify Jetson logging;
- download AnyVisLoc (public release) and check its licence;
- decide the HGM go/no-go.

Exit criteria:

- AerialVL usable;
- ≥20 query/map pairs visualized;
- one query → metric fix works;
- the simulator runs on one sequence;
- Jetson works.

Month 2 — Stable localization baseline

Tasks:

- freeze retrieval, matcher and verification;
- validation errors;
- log features;
- tile-level cache (or fixed offset-grid fallback) for offset/search-radius conditions, **train/validation sequences only**.

Exit criteria:

- fixed pipeline;
- localization table (validation sequences);
- fix records for **train/validation sequences only**; none for test sequences.

Month 3 — Reliability baselines and calibration

Tasks:

- raw, SatLoc-style, chi-square-consistency and calibrated (both variants) models;
- calibration metrics;
- leakage audit.

Exit criteria:

- frozen calibration and gate thresholds from validation data;
- reliability plots.

Month 4 — Sequential simulator (validation sequences)

Tasks:

- drift regimes and filter mis-specification;
- all M10 policies;
- offset-dependent fix selection;
- call-rate sweep.

Exit criteria:

- complete simulator;
- identical-seed comparisons on validation sequences;
- first error-vs-call-rate plots.

Month 5 — Reacquisition, Jetson, configuration freeze (validation sequences)

Tasks:

- REACQUIRE + confirmation; tune N, k, τ on validation sequences;
- Scenarios A and B on validation sequences;
- port the pipeline to the Jetson;
- profile;
- re-check calibration;
- integrate measured costs;
- **configuration freeze** (§9.1): tag the commit with all code, configs and tuned parameters.

Exit criteria:

- validation recovery results with separate REACQUIRE false-accept rate;
- Jetson table;
- freeze tag created **before** any test-sequence processing.

Month 6 — Held-out test evaluation, transfer, optional extensions

Tasks:

- generate the test-sequence tile cache / fix records **once** with the frozen code;
- run the full experiment matrix (Section 11) on test sequences, including Scenarios A and B, with frozen configurations;
- AnyVisLoc calibration transfer (frozen models; scene-disjoint re-calibration);

- appearance/viewpoint stress tests;
- optional OrthoLoC/HGM/Blender/PX4 SIH.

Exit criteria:

- held-out test results for all mandatory experiments;
- optional work may stop without harm.

Month 7 — Final validation and writing

Tasks:

- reproducibility re-runs of the frozen configuration only; no re-tuning after test results (bug fixes documented, per §9.1);
- final figures;
- archive logs/configs;
- demo;
- report;
- manuscript.

Exit criteria:

- reproducible package;
- final tables;
- demo;
- report and manuscript draft.

---

15. Risk Register and Mitigation

Risk	Impact	Mitigation
No real UAV flight	None for core	Excluded from mandatory scope
No local UAV imagery over HGM	None for core	Public benchmark is the query source
HGM delayed/unavailable	Low	Optional; AnyVisLoc aerial maps / OrthoLoC as surrogates
SatLoc data unavailable	Low	Paper used only for heuristic baseline
AerialVL unusable	Medium	UAV-VisLoc sequential fallback
AnyVisLoc download/licence issue	Low	UAV-VisLoc cross-dataset check
Retrieval/matcher too slow	Medium	Fixed subset, reduce top-K, cache descriptors
TensorRT fails	Low	PyTorch/ONNX on Jetson
Raw score already predicts errors well	Medium	Contribution becomes validated calibration + navigation consequence + reacquisition; negative result acceptable
Proposed gate only beats weak baselines	High	Mandatory chi-square and SatLoc-style baselines, combined gate, randomized/label-oracle references
Chi-square/prior consistency rejects correct large-drift fixes	High	Mahalanobis in NORMAL; REACQUIRE with widened search
REACQUIRE accepts a false fix	High	Mandatory k-fix confirmation against dead-reckoned motion; separate false-accept rate
Label leakage	High	Ground-truth-only labels; leakage audit in Month 3
Thresholds overfitted to test sequences	High	Sequence-disjoint splits; test-sequence cache generated only after the Month-5 configuration freeze; single held-out run
Unfair comparison due to search windows	High	Deterministic tile-level cache (or fixed offset grid); identical seeds
Synthetic HGM results look unrealistically easy	Medium	Kept separate; never claimed as real validation
Time sunk into simulator plumbing	High	No 3D simulator mandatory
UAV leaves preloaded map coverage	High	Detect OUT_OF_MAP; continue drift-only estimate; report separately
Reference map is stale/domain-shifted	Medium	Reliability gating; map-date/provider stress tests
Map/index storage too large for Jetson	Medium	Restrict AO; hierarchical/coarse-to-fine storage; compress/precompute descriptors

---

16. Minimum, Strong, and Stretch Outcomes

Minimum — completion target

- AerialVL end to end;
- reproducible query → map fix;
- ground-truth labels at the fixed 10 m / 25 m thresholds;
- raw, SatLoc-style, chi-square and calibrated reliability evaluated;
- drift simulator with mis-specification;
- all M10 policies compared across the call-rate sweep;
- reacquisition Scenarios A and B tested;

- held-out test-sequence evaluation run once after the configuration freeze;
- Jetson run;
- report and repository.

Strong

- calibrated (or combined) reliability clearly reduces harmful corrections vs chi-square and heuristic gating at every call rate;
- label-oracle-gap and risk–coverage analysis explain when/why;
- reacquisition recovers without false accepts;
- calibration transfer to AnyVisLoc characterized;
- useful Jetson operating point identified.

Stretch

- HGM transfer / Blender views;
- OrthoLoC transfer;
- DSM-assisted verification;
- triggered scheduler;
- PX4 SIH demo;
- follow-on real flight.

---

17. Suggested Team Division

Four students:

- **Student A — Dataset/geospatial:** AerialVL/AnyVisLoc loaders; tiling/georeferencing; altitude-above-ground from DEM; HGM adapter if data arrive; coordinate validation.
- **Student B — Visual localization:** retrieval; matching; verification; offset/search-radius fix generation.
- **Student C — Reliability/navigation:** labels; SatLoc-style, chi-square and calibrated reliability; drift simulator; gates; randomized/label-oracle references; REACQUIRE + confirmation; call-rate sweep.
- **Student D — Jetson/systems:** deployment; profiling; caching/optimization; optional PX4 SIH or Blender.

Three students: merge A with B; keep reliability/navigation and Jetson roles distinct.

---

18. Expected Deliverables

1. Reproducible RGB localization baseline (AerialVL).
2. Fix-record dataset with offset/search-radius conditions.
3. Raw / SatLoc-style / chi-square / calibrated reliability evaluation.
4. Deterministic drift simulator (with filter mis-specification).
5. Call-rate sweep comparison of all gates with randomized and label-oracle references.
6. Reacquisition-with-confirmation experiment (correct and false distant fixes).
7. Cross-dataset calibration transfer (AnyVisLoc).
8. Jetson benchmark.
9. Optional HGM/synthetic study.
10. Final report and paper-style manuscript.
11. Demo video.

---

19. Priority Reading — Start Here

1. **He, M., Chen, C., Liu, J., Li, C., Lyu, X., Huang, G., & Meng, Z. (2024).** AerialVL: A Dataset, Baseline and Algorithm Framework for Aerial-Based Visual Localization With Reference Map. *IEEE Robotics and Automation Letters*, 9(10), 8210–8217.
   Why: primary dataset and baseline. Repo: `github.com/hmf21/AerialVL`.

2. **Li, C., He, M., Chen, C., Liu, J., Lyu, X., Huang, G., & Meng, Z. (2025).** GeoVINS: Geographic-Visual-Inertial Navigation System for Large-Scale Drift-Free Aerial State Estimation. *IEEE Transactions on Robotics*, 41, 5835–5853. DOI: 10.1109/TRO.2025.3613467.
   Why: close systems-level work combining map-based geographic information with long-term state estimation.

3. **Zhou, X., Zhang, X., Yang, X., Zhao, J., Liu, Z., & Shuang, F. (2025).** Towards UAV Localization in GNSS-Denied Environments: The SatLoc Dataset and a Hierarchical Adaptive Fusion Framework. *Remote Sensing*, 17(17), 3048. DOI: 10.3390/rs17173048.
   Why: the heuristic-confidence baseline.

4. **Zhang, Y., Yi, Q., Zong, Y., Yan, Z., & He, H. (2026).** Map-Aided UAV Visual Localization via Visual Odometry and Orthophoto Map Alignment. *2026 2nd International Conference on Unmanned Systems and Technology (UST)*, 72–78. DOI: 10.1109/UST70584.2026.11663724.
   Why: very close recent systems work: heuristic quality gating plus local relocalization. It is the clearest reason this project

must focus on **calibrated reliability and comparison with chi-square gating**, not merely "gate bad fixes" or "reacquire after drift."

5. **Ye, Y., Teng, X., Chen, S., Liu, L., Wang, K., Song, X., & Li, Z. (2026).** Exploring the Best Way for UAV Visual Localization under Low-altitude Multi-view Observation Condition: A Benchmark. *CVPR 2026 Findings*, 1731–1741. Repo: `github.com/UAV-AVL/Benchmark`.
   Why: AnyVisLoc; secondary dataset (current release: 20,077 images, 24 scenes) and runnable retrieval → matching → PnP baseline code.

6. **Xu, W., Yao, Y., Cao, J., Wei, Z., Liu, C., Wang, J., & Peng, M. (2024).** UAV-VisLoc: A Large-scale Dataset for UAV Visual Localization. arXiv:2405.11936. Repo: `github.com/IntelliSensing/UAV-VisLoc`.
   Why: sequential fallback.

7. **Bar-Shalom, Y., Li, X. R., & Kirubarajan, T. (2001).** *Estimation with Applications to Tracking and Navigation.* Wiley. (Chapters on validation gates / normalized innovation squared.)
   Why: the chi-square baseline.

8. **Wang, Z., Shi, D., Qiu, C., et al. (2024).** Sequence Matching for Image-Based UAV-to-Satellite Geolocalization. *IEEE Transactions on Geoscience and Remote Sensing*, 62. IEEE Xplore document 10415460.
   Why: sequence-aware alternative to frame-wise localization.

9. **Wu, R., Cheng, X., Zhu, J., et al. (2024).** UAVD4L: A Large-Scale Dataset for UAV 6-DoF Localization. *3DV 2024*, 1574–1583. DOI: 10.1109/3DV62453.2024.00156.
   Why: 6-DoF benchmark and DSM-based pipeline.

10. **Dhaouadi, O., Marin, R., Meier, J., Kaiser, J., & Cremers, D. (2025).** OrthoLoC: UAV 6-DoF Localization and Calibration Using Orthographic Geodata. arXiv:2509.18350.
    Why: optional orthophoto + DSM surrogate; verify current venue/license before relying on it.

11. **Dhaouadi, O., Bauer, Z., Meier, J. M., Wysocki, O., Pollefeys, M., & Cremers, D. (2026).** OrthoTrack: Continuous 6-DoF UAV Trajectory Estimation Anchored in Public Orthophotos. arXiv:2606.25245.
    Why: recent continuous orthophoto + DSM trajectory anchoring; especially relevant if the optional HGM/DSM direction is pursued.

12. **Ahmedelbadawi, H., Żugaj, M., González-Moreno, M. T., & Moreno Cuenca, L. (2025).** Vectorizing Earth Elevation for Image-Based UAV Localization in GNSS-Denied Environments. *Proc. SPIE*, 13680. DOI: 10.1117/12.3069590.
    Why: optional DSM/elevation extension.

13. **HGM official product documentation** (orthophoto, SYM5-L0, Göktürk-2, sample products).
    Why: re-check terms, formats and CRS at request time.

---

20. First-Week Tasks

1. Repository, environment file, issue board.
2. One-page system diagram (prior → search region → localizer → gates → filter; NORMAL/REACQUIRE).
3. Literature table with explicit novelty check (GeoVINS, SatLoc, Zhang et al. 2026 quality gating/relocalization, chi-square gating).
4. AerialVL downloaded; fields and license verified.
5. ≥20 UAV queries plotted on the reference map.
6. One query → metric map fix.
7. Common fix-record schema committed.
8. First random-walk drift trajectory.
9. M3 thresholds (10 m primary, 25 m secondary) written into the README before any tuning.
10. Chi-square gate implemented with a unit test.
11. Jetson environment + `tegrastats` logging check.
12. AnyVisLoc download started; HGM go/no-go decision written.

No ROS, PX4, Gazebo, model training or real flight in Week 1.

---

21. Final Project Positioning

> **A calibrated reliability study for map-based absolute visual fixes in GPS/GNSS-denied UAV navigation, evaluated with real UAV imagery and deterministic sequential simulation against heuristic and chi-square baselines, with explicit large-drift reacquisition evaluation, and deployed on Jetson Orin Nano.**

The project does **not** claim that map matching, Kalman fusion, confidence weighting or chi-square gating is new. Its contribution is:

- the explicit validation and calibration of fix reliability;
- evidence of whether image-based reliability adds value beyond covariance-based gating;
- the effect on sequential navigation across call rates;

- an explicit evaluation of recovery from large drift.

Turkish/HGM data are a valuable extension, but the project is designed so that their absence cannot prevent completion.